

UNIVERSITY OF PISA

COMPUTER ENGINEERING



Emergency Room IoT Infrastructure

Professors

Giuseppe Anastasi
Francesca Righetti
Carlo Vallati

Authors

Alessandro Xavier Battisti
Francesco Della Maggiora

Academic Year 2025/2026

Contents

1	Introduction	2
2	System Design	3
2.1	Triage	3
2.2	Nurse assignment	3
2.3	Architecture scheme	4
3	Components	5
3.1	MQTT vs CoAP	5
3.2	Border router	5
3.3	Patient device	6
3.3.1	Bootstrap	6
3.3.2	Patient MQTT	7
3.3.3	Patient CoAP	7
3.4	Nurse	8
3.5	Doctor application	8
3.6	Cloud application	8
3.6.1	Database	9
3.7	Machine learning model	10
4	Experimental results	12
4.1	Adaptive mechanisms	12
4.2	Measured results	12
4.2.1	Congestion Avoidance	12
4.2.2	BR intermittent behaviour	13
4.3	Conclusions	14

Chapter 1

Introduction

The goal of this project is to tackle the inefficiencies of hospital ERs.

Normally, patients get accepted in emergency rooms, visited at triage and then they are assigned a static priority. What happens next is that ill or injured people have to wait extremely long times before being treated or simply visited again.

An IoT infrastructure could be designed to improve this process. Through the utilization of sensors and actuators, coordinated by a central control logic, a better quality of life for ER patients, doctors and nurses may be achieved.

For example, queueing times could be greatly reduced, emergency situations might be spotted earlier and consequent life-saving actions could be performed sooner.

The main idea is to utilize sensors to keep track of patients' vitals in real time, and communicate this information to the cloud application. Through this knowledge, priorities can be changed dynamically and emergencies can be detected quicker. Furthermore, patients can use their device to contact nurses for assistance, who are equipped with a device of their own.

The following chapters illustrate the design choices and implementation of the system.

Chapter 2

System Design

2.1 Triage

When a patient joins the ER, the triage assessment is performed and a priority code is assigned. For the realization of this project, the Italian national triage guidelines have been followed:

Code	Color	Priority	Max waiting time
1	Red	Emergency	0 mins
2	Orange	Urgent care	15 mins
3	Blue	Delayed urgency	60 mins
4	Green	Minor urgency	120 mins
5	White	Non-urgent	240 mins

Table 2.1: Italian emergency department triage codes.

After triage, patients are assigned to one of the ER's departments and are equipped with sensors that keep track of their:

- Heart rate
- Oxygen saturation
- Respiration rate
- Temperature
- Systolic pressure
- Diastolic pressure

These measurements are sent periodically to a sink node via MQTT communication. Each department has its own border router to reach the cloud. For each vital sign an **alert range** is defined: if any vital lies beyond the security thresholds, the transmission rate increases and a specific MQTT topic is utilized.

2.2 Nurse assignment

Every patient is assigned to a nurse. Specifically, each nurse can have a max pre-defined number of patients under their responsibility. The association itself is done statically at triage, when a new patient arrives (i.e. through the doctor application). Each patient associated to a nurse gets assigned a unique color (among the other

patients of that nurse) to ease their identification.

At this point, patients can press the button on their device to call their nurse. The communication is performed via CoAP (which is mostly utilized to perform peer to peer communication) and takes place as follows:

- The patient's device LED lights up with their assigned color.
- The nurse's device stores every assistance request in an ordered queue based on the priority of the patients who issued the requests. The nurse LED shows the color of the patient with the highest priority among the current requests.

2.3 Architecture scheme

Below is a picture of the architecture main components (considering a single department):

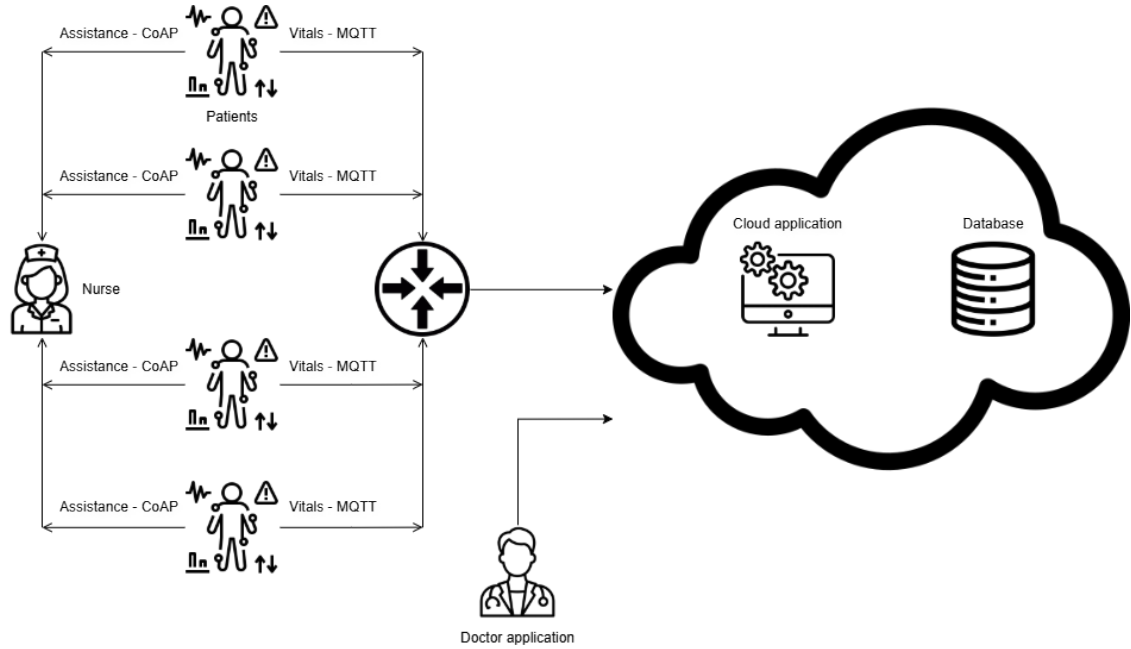


Figure 2.1: System architecture

Each actor, together with relative implementations, will be analyzed in the next chapter.

Chapter 3

Components

3.1 MQTT vs CoAP

MQTT was deemed suitable for the transmission of vital signs because of the frequency and nature of this type of communication. Sensors periodically produce measurements which have to be delivered to the cloud application, with a production rate that may increase in the presence of alerts. This communication pattern naturally maps to the publish/subscribe paradigm provided by MQTT: patient devices act as publishers, while the cloud application subscribes to the corresponding topics and consumes the measurements as they become available. This avoids the need for the cloud application to periodically poll the devices for new measurements, which would introduce unnecessary request traffic and communication overhead.

CoAP is instead used for direct communication between devices. These operations are more sporadic and typically involve an explicit request or a change in the state of one or more devices. They are naturally represented by CoAP's request/response model and its RESTful resource abstraction. Moreover, when the communicating devices are part of the same local network, direct machine-to-machine (M2M) communication allows the exchange to take place without involving the cloud application as an intermediary, reducing unnecessary communication hops and, consequently, the associated energy and network overhead.

3.2 Border router

Border routers are unique within each department and their role is to connect the network to the cloud.

Devices that run border routers have the ability to call other departments through the use of the button; Each department's BR has a pre-determined code, which can be dialled by pressing the button a certain number of times (e.g. 1 click for pediatrics, 2 clicks for obstetrics, ...).

To understand which department is calling, each one is assigned a unique LED color: when the BR of department *A* calls the BR of department *B*, the LED of *B*'s border router glows with the color of department *A*.

This functionality is implemented via the `/dept/signal` CoAP resource that's ex-

posed by the BR module, meaning that each BR acts a server by exposing this resource and also as a client by accessing this resource on other BRs.

Specifically, after the button has been pressed N times, a PUT request is sent to the N -th department with a payload containing the caller's `DEPT_ID`. If a call is received, clicking the button clears it, so the LED is turned off and that click doesn't count for a new dial. If more than one call is received, the last caller wins, so subsequent calls overwrite the previous ones.

3.3 Patient device

All devices bear a unique `DEVICE_ID`. When a patient joins the triage, they are stored in the database and a device is assigned to them. The latter will perform a bootstrap operation, and then communicate via both MQTT and CoAP for different purposes.

3.3.1 Bootstrap

At startup, devices act as CoAP clients and send a GET request to the resource `/er/patient/registration/<DEVICE_ID>`. This way they can learn all the information they need about the patient they are attached to, namely a JSON file with the following structure:

```
{
  "PATIENT_ID":      <number>,
  "TRIAGE_CODE":     <code>,
  "NURSE_ADDRESS":   <IPv6 address>
}
```

Subsequently, the device publishes on the MQTT topic `/er/patient/<PATIENT_ID>/registration` and send the following JSON file:

```
{
  "DEVICE_ID": <number>,
  "PATIENT_ID": <number>,
  "SENSORS": [
    "heart-rate",
    "spo2",
    <other sensors>
  ]
}
```

The JSON format has been deemed fitting everytime payloads are small and not composed of measurements.

3.3.2 Patient MQTT

The patient's device publishes its measurements on two different topics: `/er/patient/<PATIENT_ID>/vitals` and `/er/patient/<PATIENT_ID>/alert`. The former is published on during normal functioning, while the latter is used alongside the first one to publish those values outside of the security thresholds. Data are published using the SenML JSON format:

```
[
  {
    "n": "heart-rate",
    "u": "beat/min",
    "v": 72
  },
  {
    "n": "spo2",
    "u": "/100",
    "v": 98
  },
  <other measurements>
]
```

This format is utilized both for the `vitals` and the `alert` topics, the difference being that only critical measurements (i.e. outside of the safe range) appear in the alert payload with the tag “-alert” following the field names.

3.3.3 Patient CoAP

Patients can act both as CoAP servers and CoAP clients. The resources they expose as servers are the following:

- `/er/patient/discharge` (Cloud $\xrightarrow{\text{PUT}}$ Patient): used when a patient is discharged from the ER. The device that was being worn turns off all functionalities and waits for shutdown.
- `/er/patient/triage` (Cloud $\xrightarrow{\text{PUT}}$ Patient): used when a doctor manually changes the patient's triage code through the doctor app.

As CoAP clients, patients can press the button on their device to ask for assistance from their associated nurse (chapter 2.2). This happens by sending a POST request to the resource `/er/patient/assistance` and a JSON payload containing the `PATIENT_ID` and the request timestamp.

Additionally, an AI model runs on patient devices (section 3.7) and an autonomous decision can be done to change the patient's triage code. To achieve this, a PUT request is sent to the cloud application resource `/er/patient/triage-report`. Within its JSON payload are the `PATIENT_ID` and the `TRIAGE_CODE`.

3.4 Nurse

All nurses are equipped with a device which stores information about associated patients.

Associated patients are stored in an array, while assistance requests get stored in an ordered queue. Higher priority patients issue higher priority requests. When two patients with the same triage code issue a request, the oldest one has a higher priority.

Nurse devices only act as CoAP servers, both to communicate with patients and to be updated by the cloud applications, and they expose these resources:

- `/er/patient/assistance` (Patient $\xrightarrow{\text{POST}}$ Nurse): patients use this resource to ask for assistance. They send a JSON payload containing their `PATIENT_ID` and the request timestamp, nurses answer with a JSON containing the color that was assigned to that patient (so that their LED can turn on). An observable resource `er/patient/assistance/status` is used to notify patients that a request has been acknowledged.
- `/er/patient/association`, `er/patient/discharge` (Cloud $\xrightarrow{\text{POST}}$ Nurse): these resource allow the cloud to inform the nurse about newly associated or discharged patients.
- `/er/patient/triage-update` (Cloud $\xrightarrow{\text{POST}}$ Nurse): used to notify a nurse that a certain patient's triage code has been changed, the payload is a JSON with the `PATIENT_ID` and the new `TRIAGE_CODE`.

3.5 Doctor application

Doctors have access to a Java application that lets them:

- Add patients to the database
- Visit patients (i.e. update their last visit timestamp)
- Discharge patients (i.e. remove them from the DB and the associated nurses' arrays)
- Update their triage code

The last two operations are performed via the previously analyzed CoAP resources. When a doctor selects the “next patient”, patient's Grafana URI (containing their vitals and alerts) is printed.

3.6 Cloud application

The cloud application is the storage unit of our design. The cloud must memorize all the sent MQTT messages, memorize the information about patients and their history (triage code evolution and vitals/alerts over time). The cloud also applies a

small control logic, coordinating the CoAP communications and the exchange between doctors and patients.

These are the CoAP resources exposed by the cloud application:

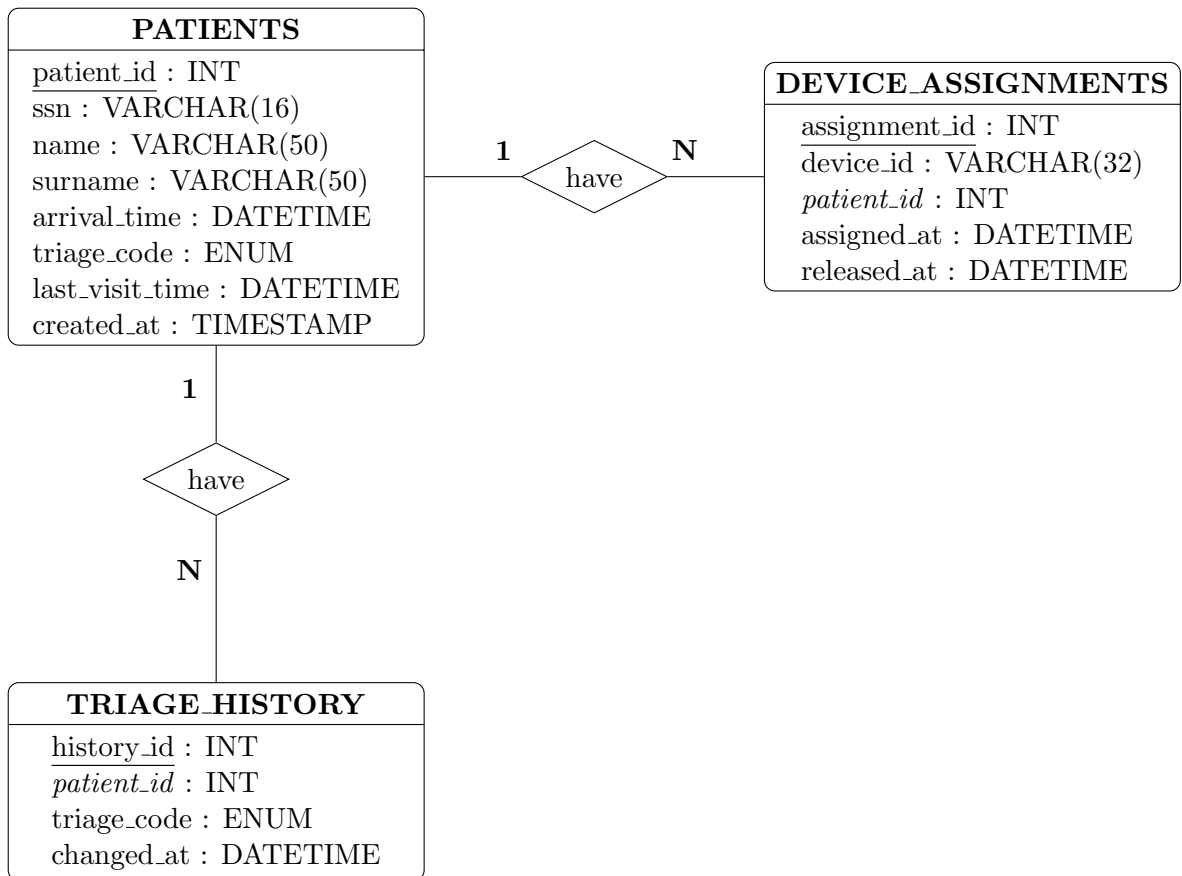
- `/er/patient/registration/<DEVICE_ID>` (Patient $\xrightarrow{\text{GET}}$ Cloud): see section 3.3.1
- `/er/patient/triage-report` (Patient $\xrightarrow{\text{PUT}}$ Cloud): see section 3.3.3

The previous sections already explain the cloud application CoAP client behaviour.

3.6.1 Database

Due to the application nature, two databases are needed.

A MySQL DB is used to store the personal information and triage history of each patient. Below is the entity-relationship diagram of the database:



To memorize vitals and alerts an Influx DB is used, since its design allows to easily store time sequences. Only one bucket “vitals” was created:

INFLUXDB BUCKET: vitals
Measurement: vitals
Tags:
patientId
source
Timestamp:
timestamp (ms)
Fields:
heartRate
spo2
temperature
pressureSystolic
pressureDiastolic
resprationRate

3.7 Machine learning model

In each patient device an MLP (Multi Layer Perceptron) model is present. This way, devices can autonomously modify the triage code during the patient's permanence. To do this, a 3-layer architecture was used, as shown in figure 3.1. The network was trained over the VitalDB dataset, a public vital signs dataset, labeled using the NEWS2 score.

After training was completed, the emlearn library allowed to convert the model into C99.

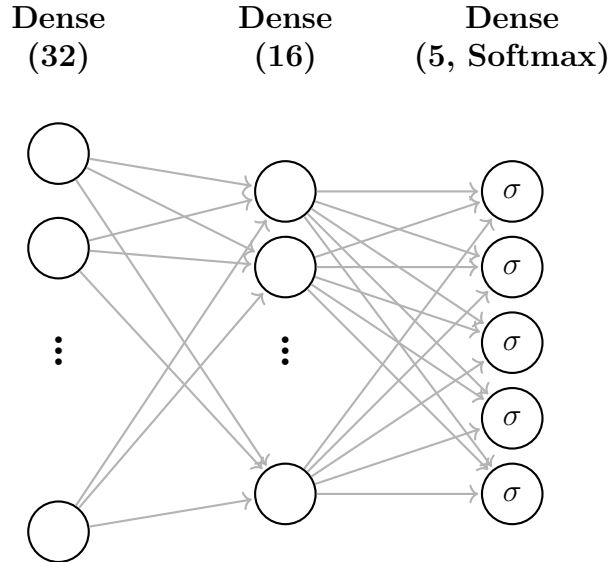


Figure 3.1: Three-layer architecture

The classifier doesn't operate on a single instantaneous reading, but rather on a sliding window over the last 20 samples. Each sample is composed of all the six measured metrics, so the window is a 20×6 matrix memorized on the device as a circular buffer.

For each classification the raw samples are inputted to a feature extractor, which summarizes each signal into five statistics: minimum, maximum, mean, standard deviation and least-square scope. This way, we were able to create a model having the same input layer for each windows size.

Figure 3.2 shows how the main indexes vary based on window size. Based on the (similar) results, size 20 was chosen. Figure 3.3 shows the normalized confusion matrix of the chosen architecture.

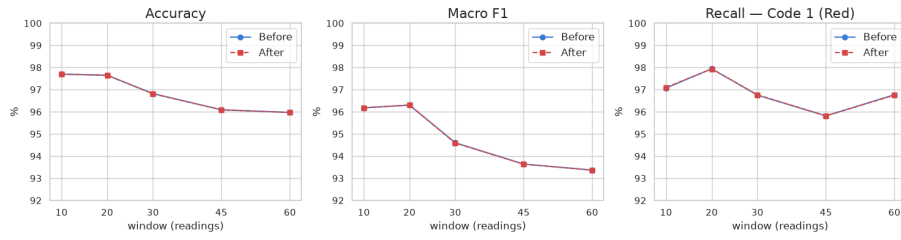


Figure 3.2: Main indexes with varying window sizes
Due to its importance, only the red code recall is shown

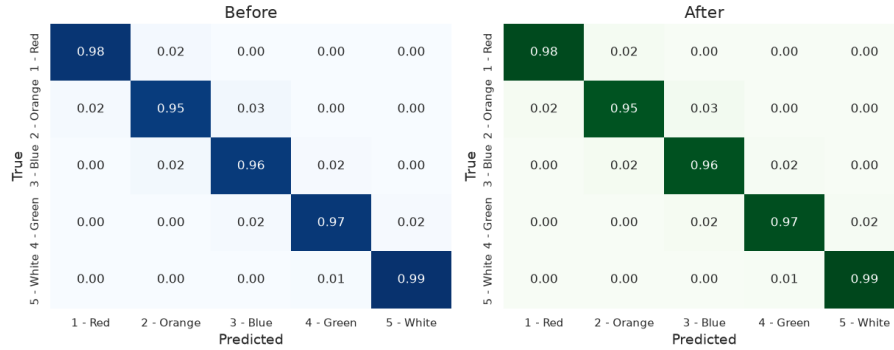


Figure 3.3: Normalized confusion matrix before and after the contiki translation

Chapter 4

Experimental results

4.1 Adaptive mechanisms

Two adaptive mechanisms were implemented, which cover the following scenarios:

- **Congestion** → white and green code vital signs stop being transmitted.
- **BR intermittent behaviour** → vitals and alerts are stored in circular buffers and are sent as soon as connectivity is restored.

Congestion is detected when either N packets in a row are lost (and it's deemed over when M packets in a row are correctly transmitted with $N < M$) or too high of a latency is measured.

A BR fault is detected by checking the RPL routing table. The gateway must be un-reachable for X times in a row to consider it offline, and then reachable for Y times in a row to consider it back online, with $X > Y$.

4.2 Measured results

4.2.1 Congestion Avoidance

To verify the effectiveness of the congestion avoidance mechanism, an experiment was run using 5 nodes: one border router and four patient devices, two with a red code and two with a white code.

The measured values were: **alerts roundtrip time (RTT)**, **alerts loss percentile** and **number of correctly received vitals**. Each experiment lasted 30 minutes and was repeated 4 times in independent conditions.

Figure 4.1 shows results for the alert messages, while figure 4.2 shows the delivery of vitals.

The mechanism impact is not very high as far as alerts are concerned, but the same cannot be said for vitals results, for which the benefit is obvious.

In fact, the number of received vitals is much higher, as lower congestion results in fewer collisions.

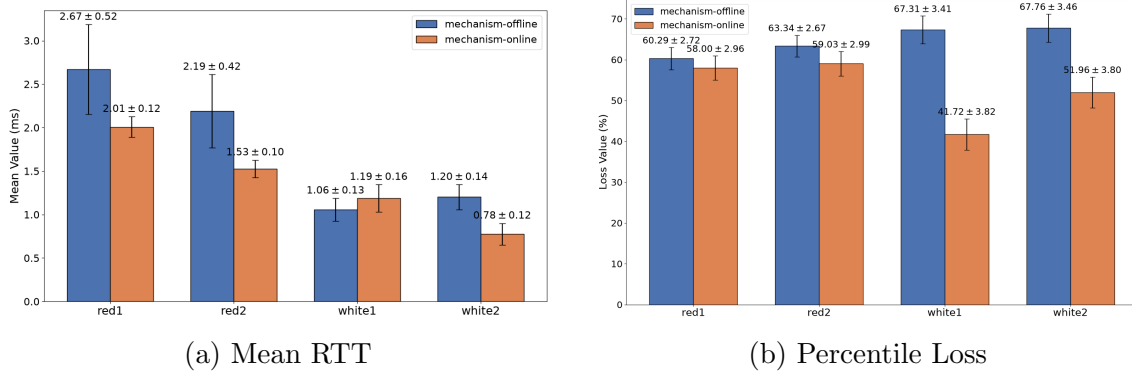


Figure 4.1: Alert messages results

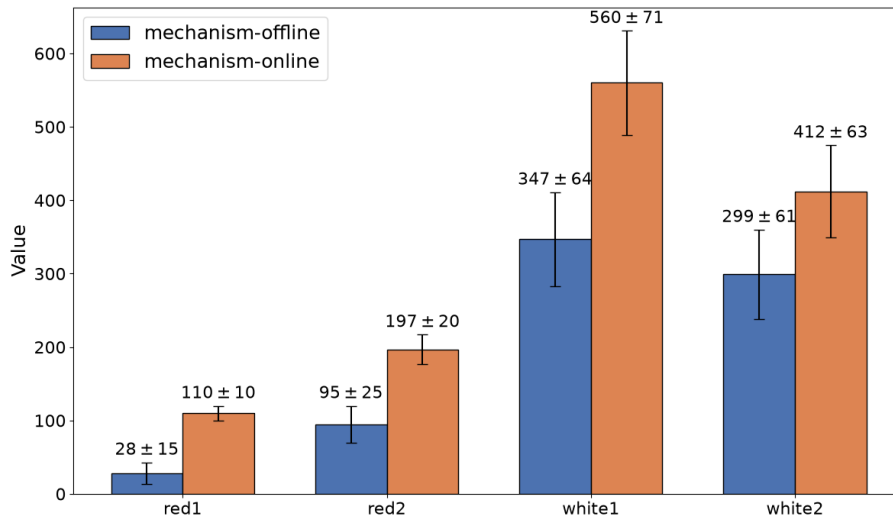


Figure 4.2: Number of vitals that successfully reach the cloud

4.2.2 BR intermittent behaviour

These were the four metrics of interest:

- *Failure detection time*: time between the failure of the gateway and the instant a node realizes it.
- *Saturation time*: time needed to fill at least one buffer.
- *Recovery detection time*: time between the re-start of the gateway and the instant a node realizes it.
- *Backlog processing time*: time required to process the backlog (i.e. empty the buffers once connectivity is restored).

Obviously, in this case it made no sense to compare the system with this mechanism turned off, as packets simply get dropped.

Measurements were performed using one white code patient device and one red code patient device (one at a time) and the experiment was repeated three times in independent conditions. Table 4.1 shows the results.

Triage color	Failure detection time	Saturation time	Recovery detection time	Backlog processing time
White	$46 \pm 25\text{s}$	$7 : 52 \pm 0 : 26\text{min}$	$20 \pm 7\text{s}$	$60 \pm 5\text{s}$
Red	$28 \pm 17\text{s}$	$7 : 10 \pm 0 : 40\text{min}$	$22 \pm 9\text{s}$	$66 \pm 9\text{s}$

Table 4.1: Intermittent BR experiment results

4.3 Conclusions

Congestion adaptive mechanism: this functionality doesn't impact alerts very much, which is to be expected as their handling doesn't change.

What does change, however, is the vital packets delivery ratio: the mechanism allows the system to detect congestion and avoid transmitting packets during it, which decreases loss and increases the overall number of received packets.

Faulty BR: the two circular buffers allow not to have complete packet loss when the BR becomes faulty, and permits at least part of the vitals and alerts to be sent as soon as the gateway comes back online (as many as the buffers can hold).